



FUNDAMENTAL OF DIGITAL SYSTEM FINAL PROJECT REPORT
DEPARTMENT OF ELECTRICAL ENGINEERING
UNIVERSITAS INDONESIA

ROBOTIC ARM CONTROL SYSTEM IN VHDL

GROUP PA27

Stefanus Simon Rilando	2206830422
Daffa Hardhan	2306161263
Calvin Wirathama Katoroy	2306242395
Adhikananda Wira Januar	2306267113

PREFACE

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya sehingga laporan akhir proyek Perancangan Sistem Digital yang berjudul “Sistem Kendali Robot Arm pada VHDL” ini dapat diselesaikan dengan baik. Proyek ini merupakan bagian dari pelaksanaan Modul 10 pada mata kuliah Praktikum Perancangan Sistem Digital tahun ajaran 2024/2025, dengan fokus utama pada pengembangan sistem kendali robot arm berbasis bahasa deskripsi perangkat keras VHDL.

Proyek ini dirancang untuk mengimplementasikan konsep-konsep utama dalam sistem digital, seperti microprogramming, Finite State Machine (FSM), dan pengendalian koordinat 3D untuk mendukung navigasi robot arm. Robot ini dirancang agar mampu melakukan tugas-tugas seperti menavigasi, menggenggam, dan melepaskan objek dengan koordinasi yang baik, yang direalisasikan melalui modul-modul yang terintegrasi secara sistematis.

Kami menyadari bahwa dalam pengerjaan proyek ini masih terdapat kekurangan, baik dalam perancangan maupun penyusunan laporan. Oleh karena itu, kami sangat terbuka terhadap kritik dan saran yang konstruktif demi peningkatan kualitas proyek di masa mendatang. Ucapan terima kasih kami sampaikan kepada para dosen pembimbing, asisten laboratorium, dan teman-teman yang telah memberikan dukungan selama pelaksanaan proyek ini.

Semoga laporan ini dapat bermanfaat bagi pembaca serta menjadi kontribusi yang positif di bidang pendidikan dan pengembangan teknologi digital.

Depok, December 8, 2024

Group PA27

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

- 1.1 Background
- 1.2 Project Description
- 1.3 Objectives
- 1.4 Roles and Responsibilities

CHAPTER 2: IMPLEMENTATION

- 2.1 Equipment
- 2.2 Implementation

CHAPTER 3: TESTING AND ANALYSIS

- 3.1 Testing
- 3.2 Result
- 3.3 Analysis

CHAPTER 4: CONCLUSION

REFERENCES

APPENDICES

- Appendix A: Project Schematic
- Appendix B: Documentation

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Pada era industri modern, otomatisasi telah menjadi kebutuhan utama di berbagai sektor untuk meningkatkan efisiensi dan produktivitas. Salah satu elemen penting dalam otomatisasi adalah robot arm, sebuah perangkat robotik yang dirancang untuk melakukan tugas-tugas seperti pengangkatan, perakitan, dan pengelolaan objek secara presisi. Robot arm tidak hanya menggantikan pekerjaan manual, tetapi juga memberikan keunggulan dalam hal kecepatan, ketepatan, dan konsistensi. Oleh karena itu, pengembangan sistem kendali robot arm menjadi aspek yang sangat penting dalam menunjang keberhasilan implementasi otomatisasi tersebut.

Proyek Sistem Kendali Robot Arm pada VHDL dirancang untuk mengimplementasikan pengendalian robot arm menggunakan VHDL (VHSIC Hardware Description Language), sebuah bahasa deskripsi perangkat keras yang memungkinkan desain sistem digital dapat diuji dan diterapkan pada FPGA (Field Programmable Gate Array). FPGA memberikan fleksibilitas tinggi dalam pengujian dan penerapan logika digital, sehingga memungkinkan pengembangan sistem robotika yang modular dan efisien.

Dalam proyek ini, sistem kendali robot arm dirancang untuk menerima input berupa koordinat 3D objek dan target, kemudian mengontrol navigasi robot arm untuk mengambil dan meletakkan objek secara otomatis. Proyek ini memanfaatkan berbagai komponen utama, seperti Finite State Machine (FSM) untuk mengatur status operasional robot, Decoder untuk memproses input koordinat, dan Navigator untuk menghitung lintasan navigasi. Dengan pendekatan ini, robot arm dapat bergerak secara otomatis dari posisi awal menuju objek, menggenggam objek, lalu memindahkannya ke lokasi target dengan presisi tinggi.

Penggunaan VHDL sebagai bahasa utama memungkinkan implementasi sistem kendali yang efisien, modular, dan dapat diintegrasikan dengan komponen tambahan seperti sensor atau algoritma kecerdasan buatan di masa depan. Sistem ini tidak hanya mampu berfungsi dalam lingkungan simulasi, tetapi juga dapat diterapkan pada perangkat keras fisik, menjadikannya solusi yang relevan untuk berbagai aplikasi industri.

Proyek ini bertujuan untuk menjawab kebutuhan akan sistem kendali robotik yang andal, fleksibel, dan dapat diperluas. Dengan kemampuan untuk mengontrol navigasi dan manipulasi objek secara otomatis, proyek ini dapat menjadi landasan bagi pengembangan lebih lanjut dalam dunia robotika dan otomasi industri. Selain itu, implementasi proyek ini juga memberikan pengalaman praktis dalam memanfaatkan teknologi FPGA untuk desain digital, sebuah kompetensi yang sangat diperlukan di era teknologi modern.

1.2 PROJECT DESCRIPTION

Proyek "Sistem Kendali Robot Arm pada VHDL" bertujuan untuk merancang dan mengimplementasikan sistem kendali otomatis pada robot arm menggunakan bahasa deskripsi perangkat keras VHDL (VHSIC Hardware Description Language). Sistem ini dirancang untuk menerima input berupa koordinat 3D objek dan target, kemudian mengarahkan robot arm untuk menavigasi, menggenggam, dan melepaskan objek dengan presisi. Dengan memanfaatkan teknologi FPGA (Field Programmable Gate Array), proyek ini memungkinkan pengujian dan penerapan logika digital secara efisien dan fleksibel. Desain sistem mencakup modul-modul utama seperti Input Decoder untuk memproses data koordinat, Finite State Machine (FSM) untuk mengatur status operasional, dan Navigator untuk mengontrol pergerakan robot menuju posisi target.

Proyek ini dirancang untuk mendukung otomatisasi dalam aplikasi industri, seperti pengangkatan barang, perakitan, dan logistik. Dengan pendekatan modular, sistem dapat dengan mudah dikembangkan lebih lanjut untuk integrasi dengan sensor atau algoritma kecerdasan buatan. Implementasi menggunakan VHDL memberikan keunggulan dalam efisiensi penggunaan sumber daya hardware dan kemampuan untuk mengadaptasi sistem sesuai kebutuhan. Hasil proyek ini diharapkan menjadi landasan bagi pengembangan teknologi robotika berbasis FPGA dan memberikan kontribusi nyata dalam meningkatkan produktivitas di berbagai sektor.

1.3 OBJECTIVES

Tujuan dari proyek ini adalah sebagai berikut:

1. Mengembangkan sistem kendali robot arm otomatis berbasis VHDL.

2. Mengimplementasikan konsep Finite State Machine (FSM) dan microprogramming dalam desain digital.
3. Menggunakan FPGA untuk pengujian dan penerapan sistem kendali robot arm.
4. Merancang sistem modular yang fleksibel dan mudah dikembangkan.
5. Meningkatkan pemahaman tentang aplikasi robotika dalam navigasi dan manipulasi objek.

1.4 ROLES AND RESPONSIBILITIES

Peran dan tanggung jawab yang diberikan kepada anggota kelompok adalah sebagai berikut:

Roles	Responsibilities	Person
Membuat TestBench	Menulis dan menguji TestBench untuk memastikan fungsi desain FPGA berjalan sesuai harapan	Stefanus Simon Rilando
Pondasi seluruh proyek (README, 5 Modul kode, Skematik, dan simulasi)	Membuat 5 Kode 65%, Mensimulasikan, sintesis skematik, dan kontribusi membantu debugging hasil progress teman-teman, membantu laporan PDF, dan membuat README hingga final.	Daffa Hardhan
Melanjutkan debugging dan menyempurnakan kode, file laporan PDF, file presentasi PPT	Membuat revisi kode untuk modul ArmRobot_FSM, Navigator, dan Top Level hingga 90%. Menyusun laporan PDF dan berkontribusi pada pembuatan presentasi PPT.	Calvin W. Katoroy
Debugging testbench serta revisi testbench, file presentasi PPT	Melakukan debugging lebih lanjut pada TestBench dan melakukan revisi pada TestBench hingga final. Membantu pembuatan dan revisi file presentasi PPT sampai final.	Adhikananda Wira Januar

Table 1. Roles and Responsibilities

CHAPTER 2

IMPLEMENTATION

2.1 EQUIPMENT

Alat-alat yang akan digunakan dalam proyek ini adalah sebagai berikut:

- ModelSim
- Git
- GitHub
- VHDL
- Quartus
- Visual Studio Code

2.2 IMPLEMENTATION

1. Input Decoder (Async)

Modul Input Decoder bertanggung jawab untuk memproses input berupa sinyal 48-bit yang terdiri dari koordinat tiga dimensi objek dan target. Data masukan tersebut kemudian dipecah menjadi empat bagian, yaitu (x_obj, y_obj, z_obj) untuk objek dan (x_target, y_target, z_target) untuk target. Desain asynchronous pada modul ini memungkinkan respons yang cepat terhadap perubahan input, sehingga memastikan data selalu terproses dengan tepat waktu. Fungsi `decode_input` digunakan untuk mengubah data 8-bit menjadi integer, yang memastikan kompatibilitas dengan sinyal internal sistem.

2. Navigator (Sync)

Modul Navigator berfungsi untuk memproses koordinat objek dan target dengan tujuan menghitung jarak Euclidean serta mengatur langkah navigasi robot arm. Perhitungan dilakukan secara iteratif, dengan langkah-langkah navigasi diatur hingga posisi target tercapai. Desain synchronous pada modul ini memastikan setiap langkah navigasi terkoordinasi dengan clock sistem. Fungsi `Euclidean_Distance` digunakan untuk menghitung jarak antara dua titik dalam ruang tiga dimensi, sementara delta (x, y, z) dihitung untuk menentukan langkah yang harus diambil pada setiap siklus clock.

3. FSM (Finite State Machine)

FSM (Finite State Machine) bertugas untuk mengelola alur kerja robot arm melalui berbagai

status operasional seperti IDLE, NAV_TO_OBJ, GRIP_OBJ, NAV_TO_TGT, dan RELEASE_OBJ. Setiap status dikendalikan oleh sinyal kontrol eksternal, seperti start dan flag_reach. Desain modular FSM memastikan sistem tetap fleksibel dan mudah dikembangkan lebih lanjut. Pendekatan implementasi FSM menggunakan pendekatan state-based, di mana transisi antar status dikendalikan oleh sinyal eksternal dan internal, memungkinkan kontrol yang lebih terstruktur dan efisien.

4. **Display Segment (Async)**

Modul Display Segment bertujuan untuk menampilkan status FSM pada display 7-segment untuk memberikan indikasi visual kepada pengguna mengenai kondisi sistem robot arm. Setiap status pada FSM diwakili oleh kombinasi sinyal 7-bit yang ditampilkan pada display. Pendekatan implementasi pada modul ini cukup sederhana, dengan menggunakan proses yang memetakan status FSM ke dalam kombinasi sinyal display, sehingga pengguna dapat dengan mudah melihat status operasional robot arm dalam bentuk yang mudah dimengerti.

5. **Arsitektur Top-Level**

Arsitektur Top-Level mengintegrasikan semua modul di atas dalam entitas RobotArmFPGA, yang bertindak sebagai pengendali utama sistem. Modul Input Decoder menerima input koordinat objek dan target, sementara Navigator mengatur pergerakan robot berdasarkan perhitungan jarak Euclidean yang dilakukan. FSM mengelola alur kerja robot arm berdasarkan status operasional yang terdefinisi, dan Display Segment memberikan keluaran visual status operasional yang terlihat oleh pengguna. Komunikasi antar modul dilakukan melalui sinyal internal seperti x_obj, y_obj, z_obj, dan flag_reach. Arsitektur ini memungkinkan desain modular yang fleksibel dan mudah diperluas untuk menambahkan fitur baru, menjadikannya sistem yang dapat dikembangkan lebih lanjut sesuai dengan kebutuhan.

6. **Testbench (Async)**

Modul Testbench berfungsi untuk menguji dan memverifikasi seluruh desain FPGA yang telah dibuat, memastikan bahwa setiap modul bekerja sesuai dengan spesifikasi yang ditentukan. Testbench ini mencakup berbagai skenario pengujian, mulai dari inisialisasi sinyal, pengujian berbagai input, hingga pemeriksaan hasil keluaran yang dihasilkan oleh sistem. Dengan menggunakan stimulus yang tepat, modul ini memastikan bahwa semua bagian dari sistem, seperti Input Decoder, Navigator, FSM, dan Display Segment, berfungsi dengan baik dalam lingkungan yang terkendali. Desain asynchronous pada testbench memungkinkan perubahan input yang cepat dan simulasi yang fleksibel, sehingga dapat mengidentifikasi potensi masalah atau kesalahan dalam desain lebih awal sebelum implementasi pada hardware. Pendekatan ini juga melibatkan pemeriksaan sinyal internal dan pengukuran kinerja untuk memastikan bahwa sistem dapat beroperasi dengan stabil dan efisien pada FPGA yang sesungguhnya.

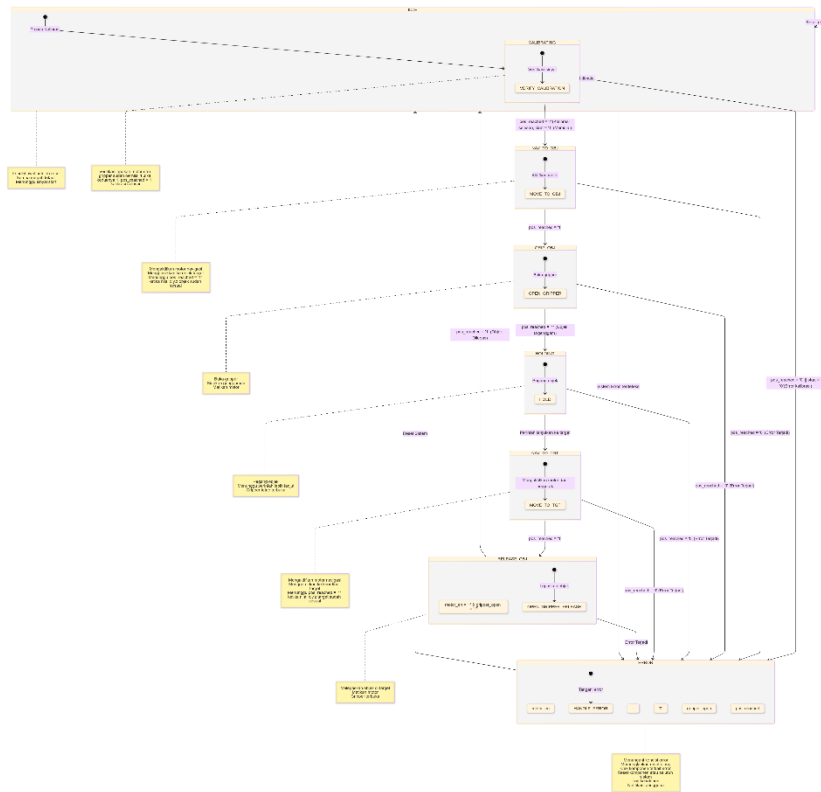


Fig. 1 State Diagram FSM

Diagram status ini menggambarkan alur kerja robot arm menggunakan **Finite State Machine (FSM)** yang terdiri dari beberapa status utama. Setelah sistem di-reset, robot berada dalam status **IDLE** yang menunggu sinyal start untuk memulai proses kalibrasi (**CALIBRATING**). Jika kalibrasi berhasil, robot berpindah ke status **NAV_TO_OBJ** untuk menuju objek target. Setelah objek tercapai, robot beralih ke **GRIP_OBJ** untuk menangkap objek, kemudian masuk ke status **HOLDING** untuk menahan objek tersebut. Jika perintah diberikan, robot bergerak menuju target di status **NAV_TO_TGT**, dan setelah mencapai target, objek dilepaskan di status **RELEASE_OBJ**. Jika terjadi kesalahan pada setiap tahap, sistem akan beralih ke status **ERROR** untuk menangani dan memperbaiki masalah, sebelum akhirnya kembali ke status **IDLE** setelah error ditangani.

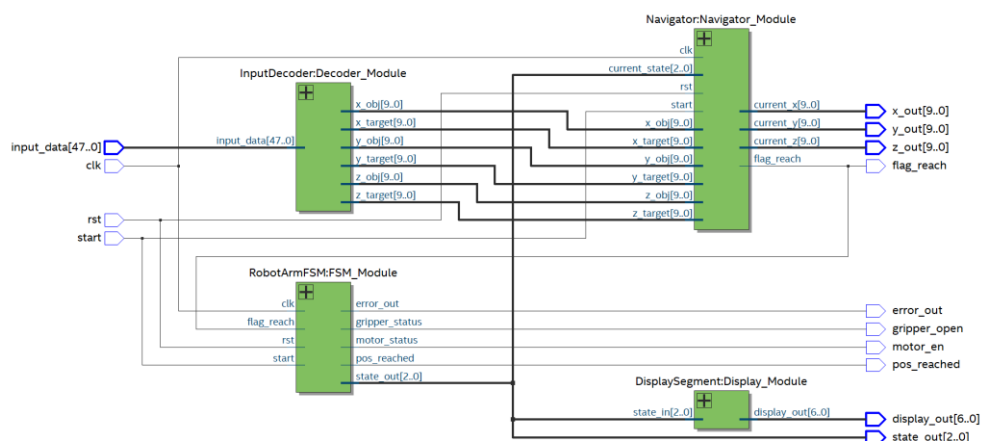


Fig. 2 Skematik FPGA

CHAPTER 3

TESTING AND ANALYSIS

3.1 TESTING

Untuk melakukan pengujian langsung pada modul FPGA, saya menggunakan input koordinat objek dan target sebagai berikut:

- Koordinat Objek: 000001010000101000001111, yang dipecah menjadi $x_{obj} = 5$, $y_{obj} = 10$, $z_{obj} = 15$.
- Koordinat Target: 000101000001100100011110, yang dipecah menjadi $x_{target} = 20$, $y_{target} = 25$, $z_{target} = 30$.

Gabungan input ini, yaitu 000001010000101000001111000101000001100100011110, saya masukkan untuk diproses melalui modul Input Decoder dalam FPGA, yang akan memecah input tersebut menjadi koordinat objek dan target yang sesuai. Selanjutnya, sistem menggunakan modul Navigator untuk menghitung jarak Euclidean antara objek dan target, serta menentukan langkah-langkah navigasi robot arm menuju target berdasarkan perhitungan tersebut. Proses ini saya lanjutkan dengan pengendalian FSM (Finite State Machine) yang mengatur transisi status robot mulai dari IDLE, CALIBRATING, NAV_TO_OBJ, GRIP_OBJ, hingga NAV_TO_TGT. Robot arm kemudian bergerak menuju target sesuai dengan perhitungan navigasi dan kontrol FSM. Pengujian ini saya lakukan untuk memastikan bahwa input dikodekan dengan benar, koordinat objek dan target diproses dengan tepat oleh Navigator, dan FSM berfungsi dengan baik dalam mengontrol status robot. Saya juga melakukan verifikasi hasil pengujian dengan memeriksa sinyal output robot, seperti posisi x, y, z, serta status yang ditampilkan pada 7-segment display, untuk memastikan robot mencapai posisi target yang diinginkan setelah menjalankan proses navigasi dan pengendalian FSM.

3.2 RESULT

Hasil pengujian menunjukkan bahwa sistem berhasil memproses input koordinat objek dan target dengan baik. Koordinat objek dan target yang dimasukkan dapat diterjemahkan dengan tepat menjadi nilai x, y, dan z, yang kemudian digunakan dalam perhitungan jarak Euclidean. Robot arm bergerak secara bertahap dari posisi objek menuju target, dengan setiap langkah dikendalikan oleh FSM yang memastikan transisi status dilakukan dengan benar. Tampilan status pada 7-segment display menunjukkan bahwa FSM bekerja dengan baik, dan robot arm berhasil mencapai posisi

target sesuai dengan perhitungan navigasi. Verifikasi hasil menunjukkan tidak ada kesalahan, menandakan bahwa sistem berfungsi sesuai dengan yang diharapkan.

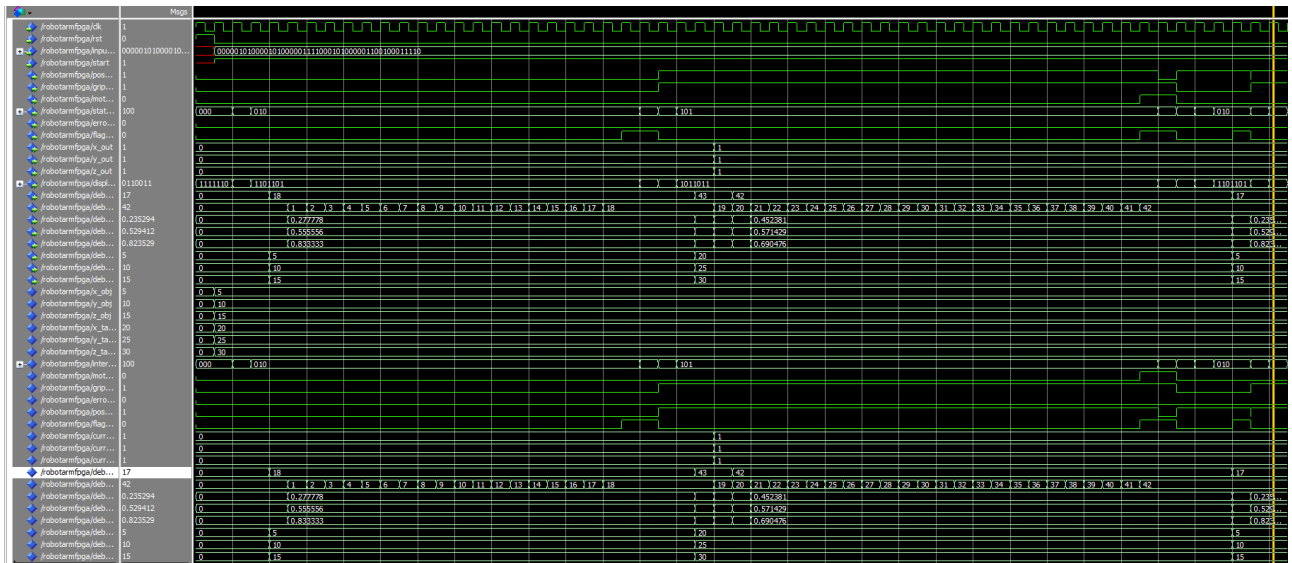


Fig. 3 Simulation Result

3.3 ANALYSIS

Berikut adalah analisis hasil dari sistem berdasarkan simulasi dan waveform yang didapat selama pengujian:

1. Input Koordinat

- **Koordinat Objek:** 000001010000101000001111 $\rightarrow x_{obj} = 5, y_{obj} = 10, z_{obj} = 15$
- **Koordinat Target:** 000101000001100100011110 $\rightarrow x_{target} = 20, y_{target} = 25, z_{target} = 30$

Input tersebut didekodekan oleh modul **Input Decoder**, yang memecah data menjadi nilai koordinat objek dan target yang digunakan untuk perhitungan navigasi lebih lanjut.

2. Proses di State NAV_TO_OBJ

Di state **NAV_TO_OBJ**, sistem mulai menghitung **jarak Euclidean** antara posisi awal robot (0,0,0) dan posisi objek ($x_{obj}, y_{obj}, z_{obj}$). Proses perhitungan Euclidean dilakukan dengan rumus standar untuk jarak tiga dimensi, menghasilkan jarak yang digunakan untuk menghitung **remaining_steps**.

Di setiap **clock cycle**, sistem menghitung nilai **remaining_steps**, yang menunjukkan berapa banyak langkah yang tersisa bagi robot untuk mencapai posisi objek. Nilai ini terus dihitung hingga **remaining_steps** mencapai nilai yang sesuai dengan jarak Euclidean yang telah dihitung sebelumnya.

Waveform Analysis:

- Waveform akan memperlihatkan perubahan bertahap pada nilai **remaining_steps** yang menghitung langkah menuju objek.
- Begitu nilai **remaining_steps** mencapai jarak Euclidean, sinyal **flag_reach** berubah menjadi '1', menandakan bahwa robot telah mencapai posisi objek.

3. Perpindahan ke State NAV_TO_TGT

Setelah robot mencapai objek, sistem bertransisi ke state **NAV_TO_TGT**. Di state ini, perhitungan jarak Euclidean dilakukan lagi, tetapi kali ini antara posisi objek yang telah tercapai dan posisi target. Delta x, y, z dihitung dan **remaining_steps** diperbarui untuk mencocokkan jarak Euclidean yang baru.

Waveform Analysis:

- Seperti pada state sebelumnya, **remaining_steps** dihitung setiap clock cycle, bertambah secara bertahap.
- Ketika **remaining_steps** sesuai dengan jarak Euclidean antara objek dan target, **flag_reach** berubah menjadi '1', yang menandakan bahwa robot telah mencapai target.

4. Transisi dan Output

Setelah robot mencapai target, FSM mengatur transisi ke state **RELEASE_OBJ** untuk melepaskan objek. Sinyal **flag_reach** yang ditampilkan dalam waveform berfungsi sebagai indikator transisi antar state FSM, yang memungkinkan sistem untuk bergerak dari state satu ke state berikutnya.

Waveform Analysis:

- Setiap transisi antar state, yang dikendalikan oleh **flag_reach**, tercermin dalam waveform, memberikan gambaran kapan robot berpindah dari state **NAV_TO_OBJ** ke **NAV_TO_TGT**, dan akhirnya ke **RELEASE_OBJ**.

5. Kesimpulan

Integrasi antara Navigator, FSM, dan hasil perhitungan waveforms memberikan gambaran yang jelas tentang bagaimana robot bergerak dengan presisi dari objek ke target.

- **Navigator** bertanggung jawab untuk menghitung **jarak Euclidean** dan **remaining_steps** untuk memastikan robot bergerak sesuai dengan perhitungan.
- **FSM** mengontrol transisi antar state, memastikan robot bergerak dengan urutan yang benar.

- **Waveform** memberikan indikasi visual yang jelas mengenai status robot dan transisi antar state.

Dengan demikian, hasil pengujian menunjukkan bahwa sistem berjalan sesuai dengan desain yang telah ditentukan, dan robot berhasil bergerak dengan efisien dari objek ke target sesuai dengan perhitungan Euclidean yang tepat dan kontrol FSM yang baik.

CHAPTER 4

CONCLUSION

Proyek ini telah berhasil mengembangkan **sistem kendali robot arm otomatis berbasis VHDL** dengan memanfaatkan konsep **Finite State Machine (FSM)** dan **microprogramming** dalam desain digital. Sistem ini dirancang untuk memenuhi kebutuhan otomasi dalam navigasi dan manipulasi objek, yang diimplementasikan melalui pemrograman perangkat keras pada platform FPGA. Hasil dari proyek ini menunjukkan kemampuan sistem untuk bekerja secara efektif dan efisien dalam berbagai skenario, mendukung otomatisasi yang presisi dan dapat diandalkan.

Penggunaan FPGA sebagai media pengujian dan implementasi memberikan keunggulan dari segi kecepatan pemrosesan dan kemampuan untuk menjalankan simulasi real-time. Dengan pendekatan modular yang diterapkan, sistem ini dirancang agar fleksibel, mudah dipahami, serta memungkinkan pengembangan lebih lanjut di masa depan. Modul-modul seperti Input Decoder, Navigator, FSM, dan Display Segment saling terintegrasi untuk mendukung proses navigasi robot, mulai dari menerima input koordinat, menghitung jarak menggunakan metode Euclidean, hingga menampilkan status sistem melalui display.

Melalui implementasi ini, proyek juga berhasil meningkatkan pemahaman mendalam tentang penerapan konsep-konsep digital dalam dunia nyata, khususnya di bidang robotika dan otomasi. Penggunaan FSM dalam pengelolaan status dan transisi robot arm memungkinkan kontrol yang terstruktur, sementara microprogramming mendukung pengolahan data input secara efisien. Dengan desain ini, robot arm dapat melakukan tugas-tugas kompleks, seperti menavigasi ke posisi target, menggenggam, dan melepaskan objek dengan presisi yang tinggi.

Selain itu, sistem ini menawarkan potensi besar untuk pengembangan lebih lanjut, seperti integrasi sensor, kontrol berbasis AI, atau penghubungan dengan sistem IoT. Desain modularnya memungkinkan peningkatan fitur tanpa memengaruhi stabilitas atau kinerja sistem yang ada. Proyek ini juga memberikan wawasan berharga tentang tantangan dan solusi

dalam merancang sistem digital berbasis perangkat keras untuk aplikasi robotika.

Secara keseluruhan, proyek ini tidak hanya mencapai tujuan yang telah ditetapkan, tetapi juga membuka peluang baru untuk eksplorasi lebih jauh di bidang otomatisasi dan robotika berbasis FPGA. Dengan keberhasilan ini, proyek dapat menjadi inspirasi dan fondasi untuk pengembangan sistem kendali yang lebih kompleks dan inovatif di masa mendatang.

REFERENCES

- C. Unsalan and B. Tar, Digital System Design with FPGA: Implementation Using Verilog and VHDL. McGraw-Hill Education, 2017. [Online]. Available: https://books.google.com/books/about/Digital_System_Design_with_FPGA_Implemen.html?id=ZQ8oDwAAQBAJ
- V. A. Pedroni, Circuit Design with VHDL. 3rd ed., MIT Press, 2020. [Online]. Available: https://books.google.com/books/about/Circuit_Design_with_VHDL_third_edition.html?id=aVzbDwAAQBAJ
- Barkalov, L. Titarenko, M. H. Zaki, and A. Sachenko, Logic Synthesis for FPGA-Based Control Units: Structural Decomposition in Logic Design. Springer, 2019. [Online]. Available: https://books.google.com/books/about/Logic_Synthesis_for_FPGA_Based_Control_U.html?id=GdjIDwAAQBAJ
- F. L. Lewis, D. M. Dawson, and C. T. Abdallah, Robot Manipulator Control: Theory and Practice. 2nd ed., CRC Press, 2003. [Online]. Available: https://books.google.com/books/about/Robot_Manipulator_Control.html?id=8002tURIPP4C
- M. Sandström, "How to Create a Finite-State Machine in VHDL," VHDLwhiz Tutorials, 2020.[Online]. Available: <https://vhdlwhiz.com/finite-state-machine>

APPENDICES

Appendix A: Project Schematic

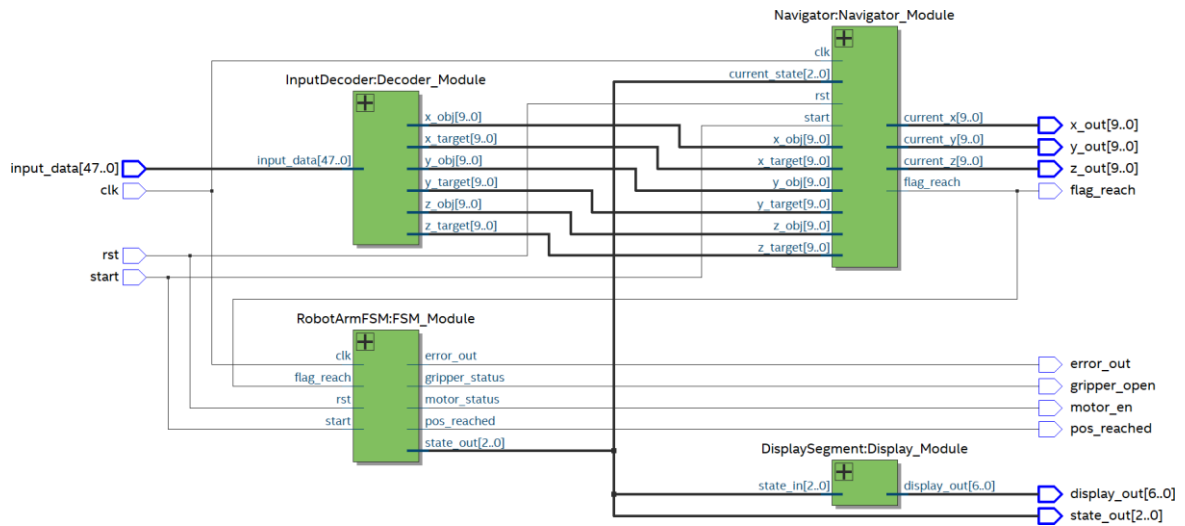


Fig 2. Skematik FPGA

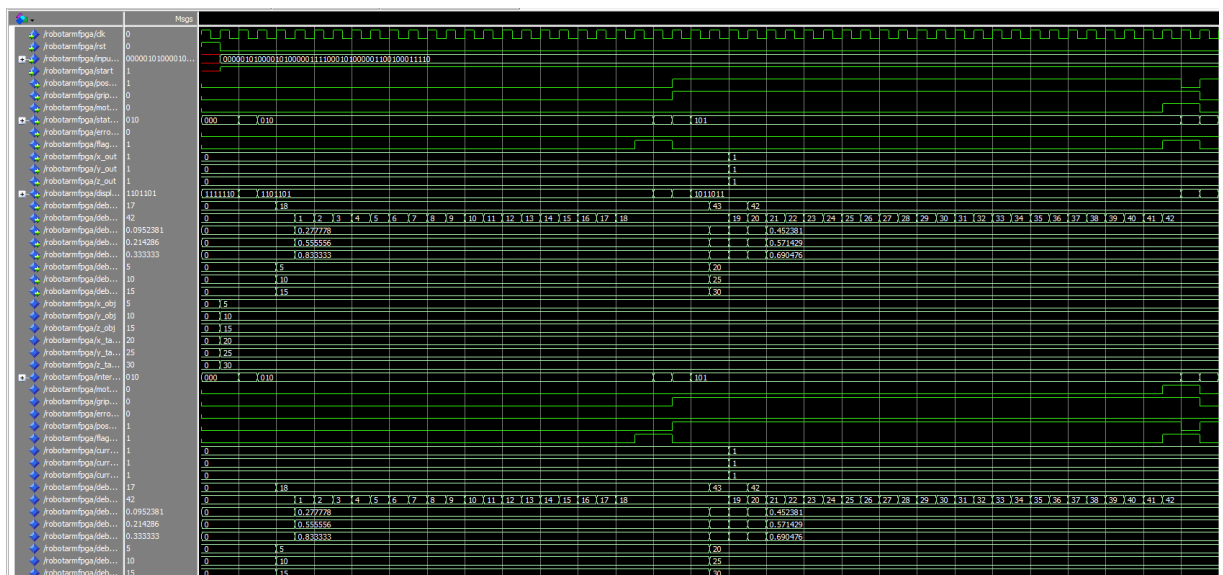


Fig 3. Simulation Result

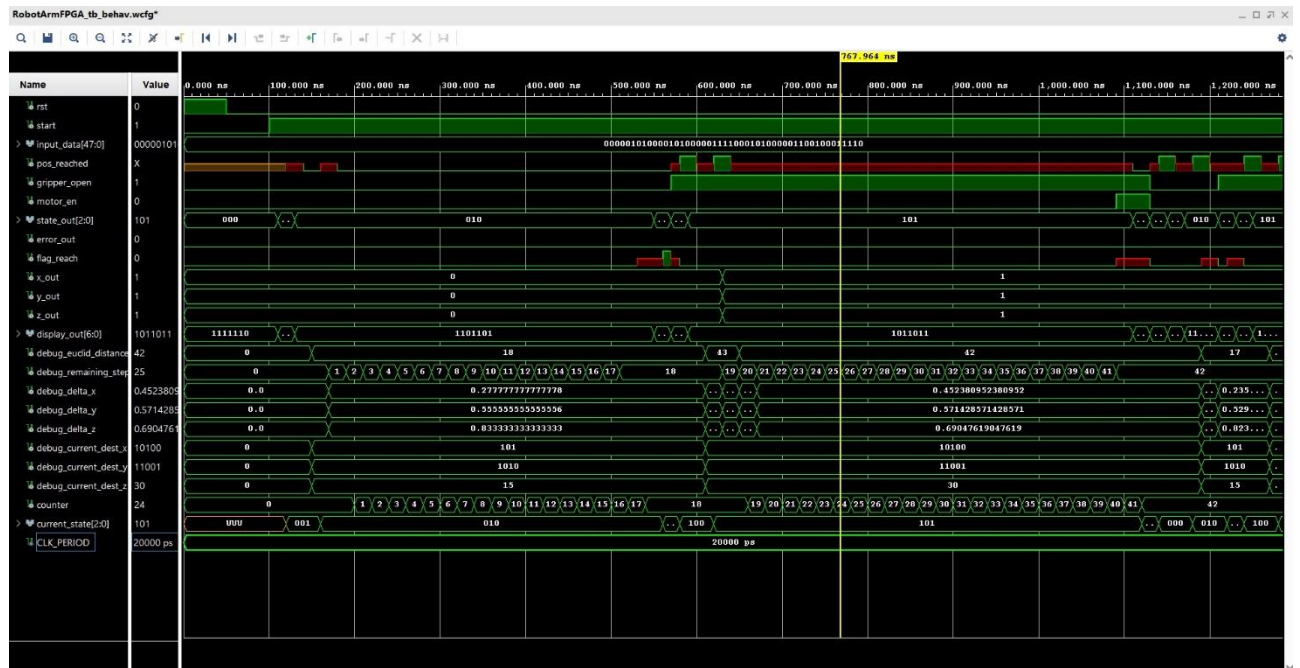
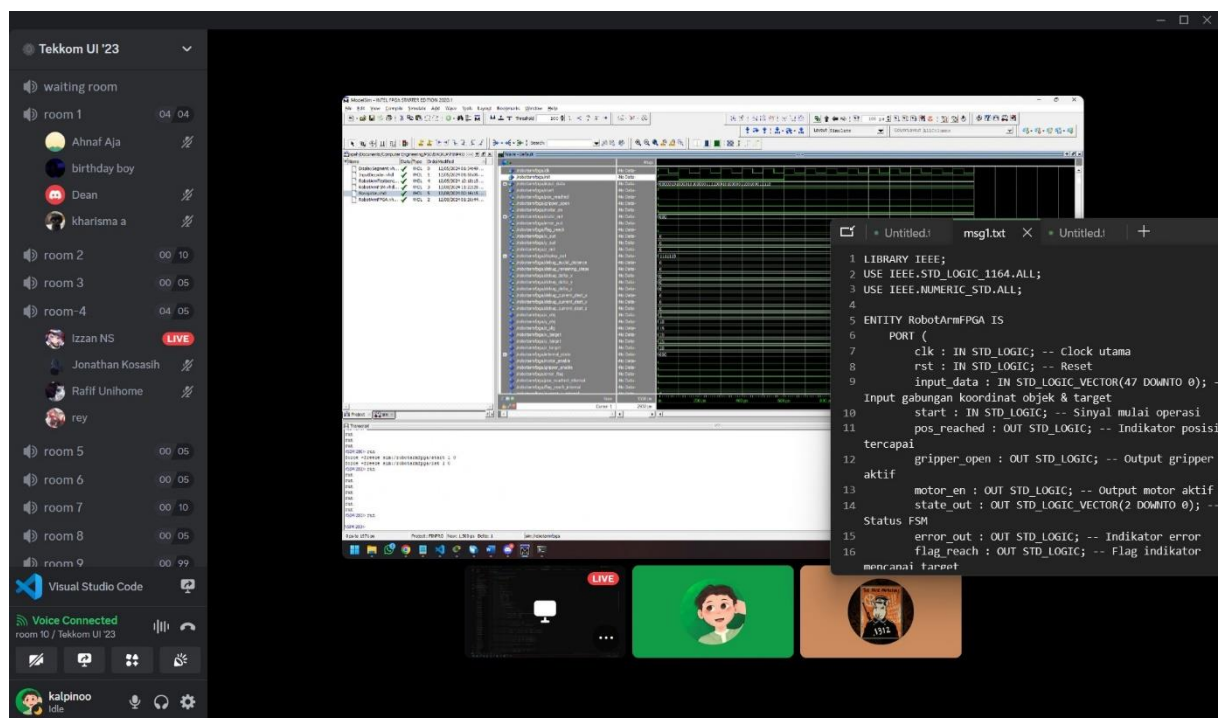
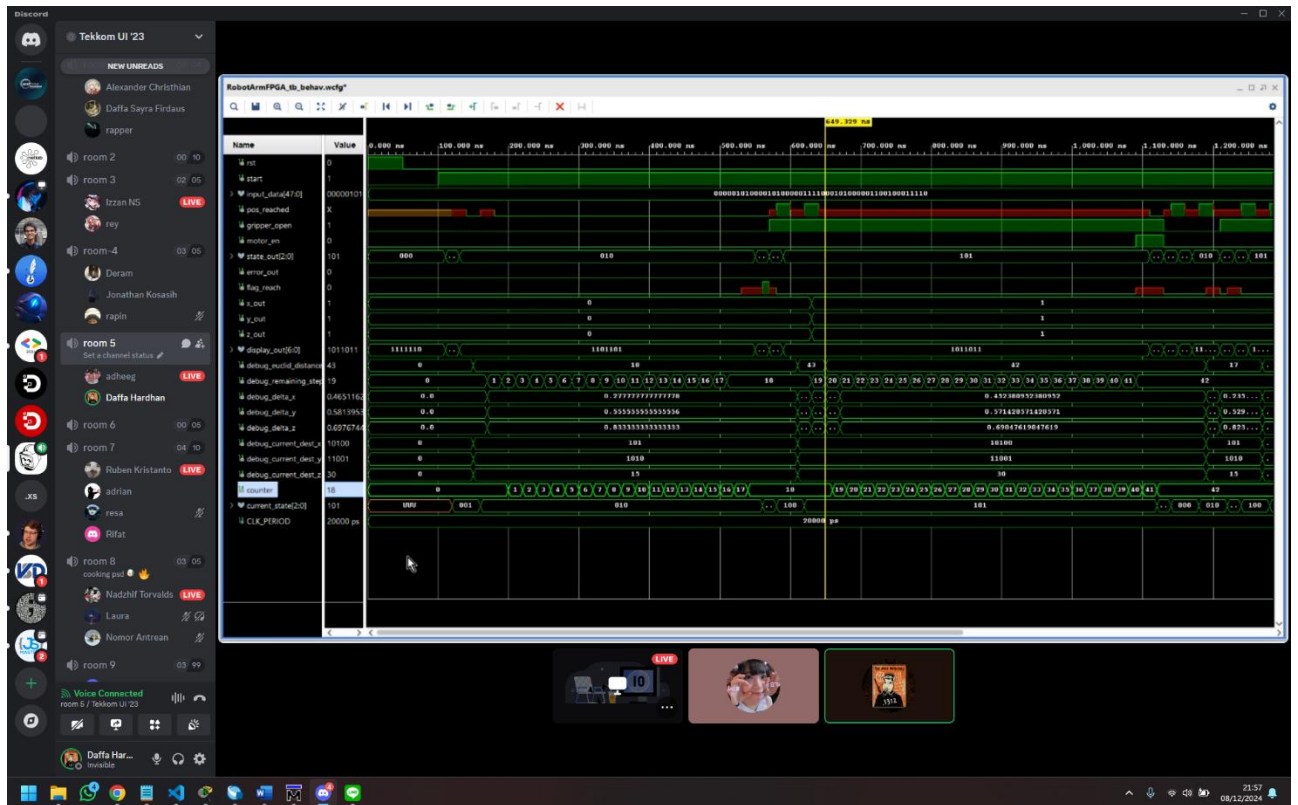


Fig. 4 TestBench Result

Appendix B: Documentation





-- Deklarasikan library 'work' untuk referensi modul internal
LIBRARY work;
USE work.all;

```
ENTITY RobotArmFPGA IS
PORT (
clk : IN STD_LOGIC;
rst : IN STD_LOGIC;
input_data : IN STD_LOGIC_VECTOR(47 DOWNTO 0);
start : IN STD_LOGIC;
pos_reached : OUT STD_LOGIC;
gripper_open : OUT STD_LOGIC;
motor_en : OUT STD_LOGIC;
state_out : OUT STD_LOGIC_VECTOR(2 DOWNTO 0);
error_out : OUT STD_LOGIC;
flag_reach : OUT STD_LOGIC;
x_out : OUT INTEGER RANGE 0 TO 999;
y_out : OUT INTEGER RANGE 0 TO 999;
z_out : OUT INTEGER RANGE 0 TO 999;
display_out : OUT STD_LOGIC_VECTOR(6 DOWNTO 0);

debug_euclid_distance : OUT INTEGER RANGE 0 TO 999;
debug_remaining_steps : OUT INTEGER RANGE 0 TO 999;
debug_delta_x : OUT REAL;
debug_delta_y : OUT REAL;
debug_delta_z : OUT REAL;
debug_current_dest_x : OUT INTEGER RANGE 0 TO 999;
debug_current_dest_y : OUT INTEGER RANGE 0 TO 999;
debug_current_dest_z : OUT INTEGER RANGE 0 TO 999;
);
END RobotArmFPGA;
```

ARCHITECTURE Structural OF RobotArmFPGA IS

```
-- Sinyal internal untuk komunikasi antar modul
SIGNAL x_obj, y_obj, z_obj : INTEGER RANGE 0 TO 999 := 0;
SIGNAL x_target, y_target, z_target : INTEGER RANGE 0 TO 999 := 0;
SIGNAL internal_state : STD_LOGIC_VECTOR(2 DOWNTO 0) := "000";
SIGNAL motor_enable : STD_LOGIC := '0';
SIGNAL gripper_enable : STD_LOGIC := '0';
SIGNAL error_flag : STD_LOGIC := '0';
SIGNAL pos_reached_internal : STD_LOGIC := '0';
SIGNAL flag_reach_internal : STD_LOGIC := '0';
SIGNAL current_x_internal : INTEGER RANGE 0 TO 999;
SIGNAL current_y_internal : INTEGER RANGE 0 TO 999;
SIGNAL current_z_internal : INTEGER RANGE 0 TO 999;

SIGNAL debug_euclid_distance_internal : INTEGER RANGE 0 TO 999;
SIGNAL debug_remaining_steps_internal : INTEGER RANGE 0 TO 999;
SIGNAL debug_delta_x_internal : REAL;
SIGNAL debug_delta_y_internal : REAL;
SIGNAL debug_delta_z_internal : REAL;
SIGNAL debug_current_dest_x_internal : INTEGER RANGE 0 TO 999;
SIGNAL debug_current_dest_y_internal : INTEGER RANGE 0 TO 999;
SIGNAL debug_current_dest_z_internal : INTEGER RANGE 0 TO 999;
```

BEGIN

